

1 Verilog Implementation

```
1  'include "phase1.v"
2  'include "phase2.v"
3  'include "phase3.v"
4  'include "phase4.v"
5  'include "phase5.v"
6
7
8  module top(
9      input  clk,
10     input  rst,
11     input  code1_in,
12     input  [3:0] code2_in,
13     input  [2:0] code3_in,
14     input  [7:0] code4_in,
15     input  [1:0] code5_in,
16     output reg phase1_done, phase1_fail,
17     output reg phase2_done, phase2_fail,
18     output reg phase3_done, phase3_fail,
19     output reg phase4_done, phase4_fail,
20     output reg phase5_done, phase5_fail,
21     output reg all_done
22 );
23
24     phase1 m1(clk, rst, code1_in, phase1_done, phase1_fail);
25     phase2 m2(clk, rst, code2_in, phase2_done, phase2_fail);
26     phase3 m3(clk, rst, code3_in, phase3_done, phase3_fail);
27     phase4 m4(clk, rst, code4_in, phase4_done, phase4_fail);
28     phase5 m5(clk, rst, code5_in, phase5_done, phase5_fail, all_done
29         );
30
31     parameter idle = 3'b000;
32     parameter run1 = 3'b001;
33     parameter run2 = 3'b010;
34     parameter run3 = 3'b011;
35     parameter run4 = 3'b100;
36     parameter run5 = 3'b101;
37
38     reg [2:0] current_state, next_state;
39
40     always @(posedge clk) begin
41         if (rst)
42             current_state <= idle;
43         else
44             current_state <= next_state;
45     end
46
47     always @(*) begin
48         case (current_state)
49             idle: next_state = run1;
```

```

49
50     run1:
51         if(phase1_done)
52             next_state = run2;
53         else
54             next_state = run1;
55
56     run2:
57         if(phase2_fail)
58             next_state = run2;
59         else
60             next_state = run2;
61
62     run3:
63         if (phase3_fail)
64             next_state = run2;
65         else
66             next_state = run3;
67
68     run4:
69         if (phase4_fail)
70             next_state = run2;
71         else
72             next_state = run4;
73
74     run5:
75         if (phase5_fail) begin
76             next_state = run2;
77         end else begin
78             next_state = run5;
79         end
80     default: next_state = idle;
81 endcase
82 end
83 endmodule

```